

Smart Cabin & Vehicle Health Monitoring System — Architecture Explanation v1.0

1. What Is Architecture — And Why It Matters

Architecture means how a system is organised. It answers three questions: What are the pieces? What does each piece do? How do the pieces talk to each other?

In this project, the architecture was designed to mirror a real car ECU (Electronic Control Unit) that runs Adaptive AUTOSAR software. That means the code is split into layers — just like a real car — and each layer only talks to the layer below it. This makes the system easy to change (fix one module without breaking the rest) and easy to test (test one module in isolation).

[!NOTE] Think of it like a factory. The **sensor floor** makes parts (sensor readings). The **assembly floor** checks quality (AlertManager, DtcManager). The **records office** files reports (Logger, EdrRecorder). The **front office** shows the status board (Dashboard, OtaSimulator). Each floor does one job and hands work to the next.

2. The Multi-Module File Structure

The entire project lives in 13 header/source pairs, heavily expanded to support 10 advanced automotive subsystems.

```
Project/
├── include/
│   ├── Sensor.hpp           ← Module 1: sensor definitions & fault injection
│   ├── Alert.hpp           ← Module 2: alert strategies & prioritization
│   ├── DtcManager.hpp      ← Module 3: diagnostic trouble codes & freeze frames
│   ├── Logger.hpp         ← Module 4: async logger & generic templates
│   ├── EdrRecorder.hpp     ← Module 5: crash-safe event data recorder
│   ├── JsonParser.hpp      ← Module 6: custom JSON parser
│   ├── Dashboard.hpp       ← Module 7: display rendering & stats
│   ├── ThreadManager.hpp   ← Module 8: safe queues, mutexes & threads
│   ├── EcuMonitor.hpp      ← Module 9: watchdog for ECM, BCM, BMS, TPMS
│   ├── OtaSimulator.hpp    ← Module 10: over-the-air firmware flash
│   ├── CanBus.hpp          ← Module 11: 11-bit CAN frame network
│   ├── DriverProfile.hpp   ← Module 12: swappable user preferences
│   └── ServiceBus.hpp      ← Module 13: SOA publish/subscribe system
├── src/
│   ├── [Matching 12 .cpp files]
│   ├── main.cpp            ← Composition root: wires everything + runs menu
│   └── tests.cpp           ← Automated boundary & integration testing
├── logs/                  ← vehicle_log.txt, crash_edr.bin, test_report.csv
├── data/                  ← config.txt, test_cases.json
└── CMakeLists.txt         ← build instructions
```

Each header file is a group — it holds all the classes that belong to one logical concern. The .cpp file for each header contains the actual code that makes those classes work.

3. The Compile-Time Include Chain

When the compiler builds the project, it needs to know about classes before it can use them. This is controlled by which file includes which. There is a strict rule in this project: the includes form a

one-way chain with no loops.

3.1 The Chain

```
Sensor.hpp      – includes NOTHING from this project (foundation)
  ↑ included by
Alert.hpp       – includes Sensor.hpp (needs it to evaluate rules)
  ↑ included by
Logger.hpp / EdrRecorder.hpp
  ↑ included by
ThreadManager.hpp – forward-declares classes to manage threads & safe queues
  ↑ included by
main.cpp        – includes all headers (it's the top-level wiring point)
```

3.2 Why No Loops

If `Alert.hpp` included `Logger.hpp`, and `Logger.hpp` included `Alert.hpp`, the compiler would try to read each file while reading the other — an infinite loop. The solution is the **forward declaration**.

```
// In ThreadManager.hpp:
class Sensor;
class AlertManager;
class VehicleStatistics;
class Dashboard;
```

`SharedVehicleData` holds pointers or references to these classes. A pointer/reference only needs to know the class exists — not what's inside it. The full `#include "Sensor.hpp"` happens only in `ThreadManager.cpp` where functions are actually called. This keeps compile times fast and avoids circular dependency crashes.

4. The Layered Architecture

The system is built in layers, each sitting on top of the one below. This is identical to how Adaptive AUTOSAR organises real automotive software.

- **LAYER 4: Infrastructure & Services (ThreadManager, Dashboard, OtaSimulator, CanBus, ServiceBus, EcuMonitor)** Shows results to the driver. Manages all 4 worker threads. Runs the SOA bus, updates firmware, simulates the CAN network, and manages ECU states.
- **LAYER 3: Persistence & Data (Logger, EdrRecorder, JsonParser)** Records events to disk asynchronously via `SafeQueue`. Buffers crash telemetry. Parses external configurations.
- **LAYER 2: Alerts & Diagnostics (Alert, DtcManager)** Detects danger using Strategy Pattern rules. Raises warnings and registers OBD-II diagnostic trouble codes.
- **LAYER 1: Foundation (Sensor, DriverProfile)** Simulates the 6 car sensors with Mersenne Twister RNG. Provides core operating thresholds. Uses NOTHING from other modules.

[!IMPORTANT] The important rule: a layer can only use layers BELOW it. Dashboard uses Sensor, Alert, and Logger. Logger uses Alert. Alert uses Sensor. Nobody reaches upward.

5. What Each Module Contains

5.1 Module 1 & 12 — Foundation

Component	What It Is	Purpose
Sensor	Abstract base	Defines polymorphic <code>update()</code> contract
EngineTemperatureSensor	Concrete sensor	Simulates engine temp 50–130°C
DriverProfile	Struct / Class	Holds thresholds (Eco/Sport/Default)

5.2 Module 2 & 3 — Diagnostics

Component	What It Is	Purpose
AlertRule	Strategy interface	Base for specific alert threshold checks
AlertManager	The alert brain	Tracks active/history with <code>priorityScore</code> engine
DtcManager	OBD-II Brain	Registers P-codes (e.g., P0117) and Freeze Frames

5.3 Module 4, 5 & 6 — Persistence

Component	What It Is	Purpose
EventLogger<T>	Template logger	Async writing without blocking sensors
EdrRecorder	Flight recorder	Circular buffer flushing to <code>crash_edr.bin</code> on impact
JsonParser	Custom parser	Parses <code>test_cases.json</code> without external libs

5.4 Module 7–11 & 13 — Infrastructure

Component	What It Is	Purpose
ThreadManager	RAII thread owner	Spawns <code>sensorThread</code> , <code>monitorThread</code> , etc.
SafeQueue<T>	Thread-safe queue	Mutex + <code>condition_variable</code> for async I/O
SharedVehicleData	Central struct	Reader-writer locked state shared across threads
CanBus	Network simulator	Packs floats into 11-bit ID / 8-

Component	What It Is	Purpose
		byte CAN frames
OtaSimulator	Flash manager	Upgrades firmware v1.0 → v2.0 safely
ServiceBus	Pub/Sub system	SOME/IP style decoupled event messaging

6. Runtime Object Ownership

When the program runs, `main.cpp` creates every object and connects them.

`unique_ptr` — Exclusive Ownership

`vector<unique_ptr<Sensor>>` is created in `main` and moved into `SharedVehicleData`. This guarantees zero copies and zero extra heap allocations while enabling polymorphic `update()` calls.

`shared_ptr` — Shared Lifetime

`shared_ptr<SharedVehicleData>` is passed to every worker thread. This keeps the centralized state (Sensors, AlertManager, Logger, etc.) alive as long as any thread is running.

RAII & Smart Synchronisation

No raw pointers, no `new/delete`. `std::shared_mutex dataMutex` allows the Dashboard to take a `shared_lock` (parallel reads) while Sensors take a `unique_lock` (exclusive writes). This ensures rendering never blocks sensor updates.

7. The Complete Data Flow

Step 1 — **Sensor Thread** updates the sensors (Module 1) Every 500ms, the RNG generates new values and applies 5% fault injection.

Step 2 — **Monitor Thread** evaluates rules (Module 2) Every 1000ms, it runs the Strategy pattern (AlertRules) against the new readings, factoring in `DriverProfile`.

Step 3 — **AlertManager & DtcManager** process danger (Module 2/3) If a threshold is breached, a high `priorityScore` Alert is raised and an OBD-II Freeze Frame is captured.

Step 4 — **Logger Thread** writes to disk asynchronously (Module 4) Alert strings are pushed into the `SafeQueue`. The Logger Thread pops them and flushes to disk — totally off the critical path.

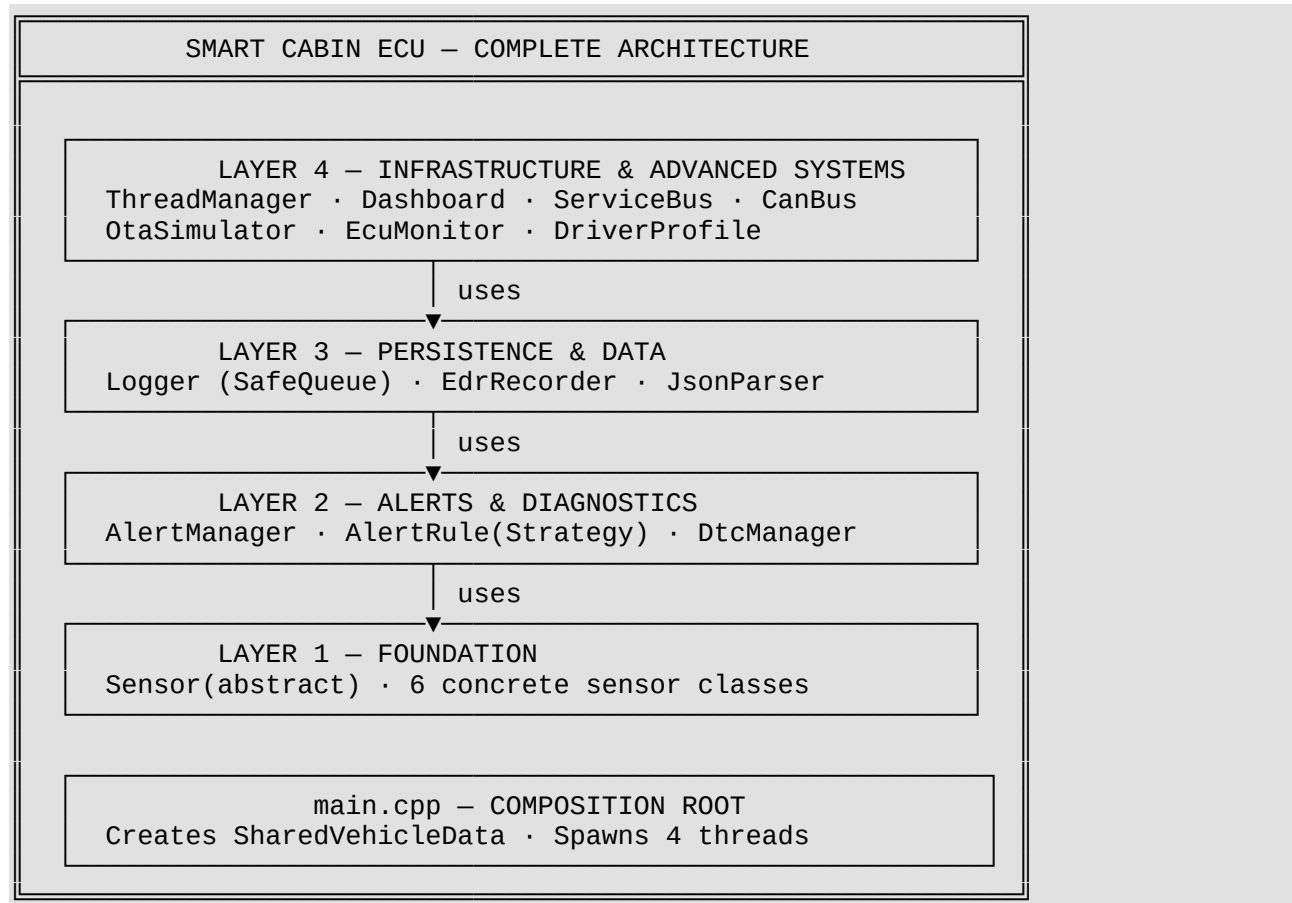
Step 5 — **Dashboard Thread** renders to screen (Module 7) Every 2000ms, it takes a `shared_lock`, reads the shared state, and prints ANSI colour-coded charts and historically mapped speed sparklines.

8. How The Modules Talk To Each Other

Each module is given references to the other modules it needs (Dependency Injection).

The Service-Oriented Communication Layer (Module 13) Beyond direct method calls, modules communicate through `ServiceBus`. This mirrors Adaptive AUTOSAR's `ara::com`. For example, the OTA simulator subscribes to `"vehicle.speed"`. The Monitor thread publishes to `"vehicle.speed"`. The OTA simulator will refuse to start flashing if `speed > 0`, yet neither module directly `#includes` the other.

9. Architecture Block Diagram



10. Automotive Mapping — Real-World Equivalents

This Project Component	Real Automotive Component	AUTOSAR Standard
Sensor Framework	MCAL (Microcontroller Abstraction Layer)	Classic AUTOSAR
AlertManager	Vehicle Health Manager (VHM)	Adaptive AUTOSAR
Logger / EdrRecorder	Cockpit Domain Controller	Classic / Adaptive
DtcManager	Diagnostic Event Manager (DEM)	AUTOSAR DEM SWC
ThreadManager	Execution Management	<code>ara::exec</code>
CanBus	CAN Communication Stack	AUTOSAR RTE
ServiceBus	SOME/IP Service-Oriented Comms	<code>ara::com</code>

This Project Component	Real Automotive Component	AUTOSAR Standard
EcuMonitor	Platform Health Management	ara::phm
JsonParser	Parameter Manager / ARXML Config	AUTOSAR ECUC
OtaSimulator	Update & Config Management (UCM)	Adaptive AUTOSAR UCM